购物车 □ 下单核心

gomall 加购 / 地址 / 雪花订单号 / 库存预扣 / DB+outbox 事务

gomall · 业务侧 deck

今日大纲

加购到下单：业务链路的三个停留点

雪花算法：64 bit 怎么切

下单是金融事件：三步必须原子

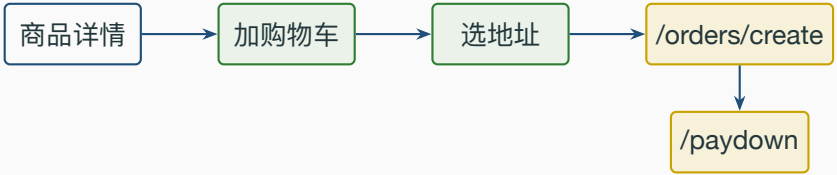
同步下单 vs 异步下单

业务码、压测数字、对账

回顾与代码路径

加购到下单：业务链路的三个停留点

下单不是一次点击，是一条业务链



- 用户视角看着一气呵成，后端是 **4 张表 + 2 个中间件 + 1 条 MQ**。
- 这份 deck 只盯 **加购 / 地址 / 下单**三步；支付走 deck 05 / deck 14。

routes/router.go:92–111 三段路由共同支撑这条链

购物车的业务必要性：不只是” 暂存”

- **凑单**：满减 / 包邮 / 跨店满减都要把多 SKU 拉到一起算价。
- **比价**：用户加 3 件同类货回头删 2 件，是真实购买决策的一部分。
- **留存**：未下单的购物车 = 召回素材，运营 7 日内发券促单。
- **合规**：跨境/酒类等 SKU 必须在结算前再做一次资质校验。

直接”Buy Now” 跳过购物车的电商也存在（Amazon 1-Click），
但需要绑定默认地址 + 默认支付 + 默认发票三件套，gomall 还没收敛到这种程度。

service/cart.go:29–87 CRUD 四件套

gomall 购物车的最小数据模型

字段	含义	业务约束
user_id	谁的购物车	索引主键的一部分
boss_id	哪个商家	跨店满减按它聚合
product_id	哪个 SKU	同一 (user, product) 只能一行
num	件数	校验 1.. 库存上限

- `ErrorProductMoreCart=20008`：重复加购同款 □ 给一个明确业务码。
- 这是为什么 `CartCreate` 调用方先 `GetProductById` 再交给 `CreateCart`。

service/cart.go:36–49 + pkg/e/code.go:31–32

地址簿：业务复杂度比想象的高

- **多地址**：家 / 公司 / 父母家，下单时由用户选。
- **默认地址**：新用户首次填的，结算页默认选中。
- **海外地址**：邮编可缺、行政区划不是省/市/区三级。
- **脏数据**：电话 11 位 vs 区号 + 座机，校验放服务端还是前端?

gomall 当前 `Address` 只保留 `name` / `phone` / `address`，
合并字段省事，但订单一旦发起就该**冻结地址快照**，
否则用户改地址后历史订单的收货地址会一起被改。*service/address.go:34–46* 当前模型；
repository/db/model/order.go AddressID 引用关系

为什么订单不用 DB 自增 ID

方案	业务问题	性能问题	
DB AUTO_INCREMENT	暴露当日单量、可枚举	单点瓶颈 / 跨库不连续	**业务侧**
UUID v4	完全随机、无序	索引页频繁分裂	
雪花算法	时间戳前缀，趋势递增	本地生成、零网络	

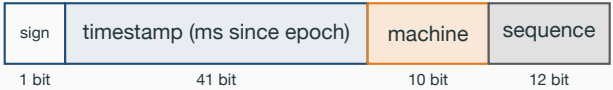
的硬约束：

- 订单号要能塞进短信 / 客服系统，不能太长（雪花 19 位十进制刚好）。
- 不能让竞争对手通过订单号 `id+1` 枚举出当日单量。
- 索引页要顺序写，避免随机 ID 把 B+ 树搅得到处分裂。

pkg/utils/snowflake/snowflake.go:10–25

雪花算法：64 bit 怎么切

雪花 64-bit 结构示意



- **41 bit 时间戳**：从 2024-01-01 起，可用 ≈ 69 年。
- **10 bit 机器位**：最多 1024 个进程，gomall 单实例够用。
- **12 bit 序列号**：单进程单毫秒最多 4096 个 ID。

pkg/utils/snowflake/snowflake.go:11 epoch = 2024-01-01 UTC

InitSnowflake：epoch 与机器位

Listing 1: *pkg/utils/snowflake/snowflake.go:10–21*

```go
func InitSnowflake(machineID int64) {
    snowflake.Epoch = time.Date(
        2024, 1, 1, 0, 0, 0, 0, time.UTC,
    ).UnixNano() / 1000000
    var err error
    node, err = snowflake.NewNode(machineID)
    if err != nil {
        panic(err)
    }
}

func GenSnowflakeID() int64 {
    return node.Generate().Int64()
}
```

逐行讲：为什么这 12 行很要紧

`Epoch=2024-01-01` 把基准从 1970 平移到 2024，腾出最高位的时间空间，41 bit 能多撑几十年。

`machineID` 部署时由配置注入，多副本必须各不相同；冲突 = 双发出现重复订单号。

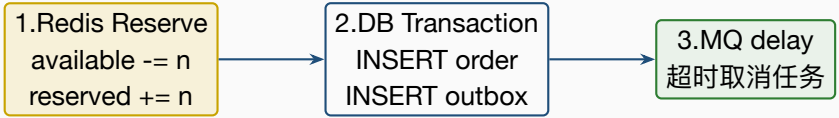
`NewNode` 失败 `panic` 启动期就挂掉，比运行期偶发生成失败更安全——订单号是金融语义，不能”先跑起来再说”。

`Generate().Int64()` 一次返回 int64，方便存 `order.order_num bigint unsigned`。

> 雪花是**本地无锁**的——OrderCreate 调用它零网络成本，这是 50K RPS 的前提。

下单是金融事件：三步必须原子

OrderCreate 拆成三段



- **第 1 步**：Redis Lua 原子 reserve，失败直接 `50001` 不进 DB。
- **第 2 步**：同一 `gorm.Transaction` 写订单 + 写 outbox，二选一不行。
- **第 3 步**：投递延迟队列，15 分钟未付款自动取消。

service/order.go:40–102 全函数 60 行

OrderCreate 主体三步

Listing 2: *service/order.go:46–86*

```go
order := &model.Order{
    UserID: u.Id,
    ProductID: req.ProductID,
    Num: int(req.Num),
    OrderNum: uint64(snowflake.GenSnowflakeID()),
}

// 1) Redis 预扣库存（事务外）
if err = cache.ReserveStock(ctx, req.ProductID, int64(req.Num)); err != nil {
    return nil, err
}

// 2) 同事务写订单 + 写 outbox
err = dao.NewDBClient(ctx).Transaction(func(tx *gorm.DB) error {
    if e := dao.NewOrderDaoByDB(tx).CreateOrder(order); e != nil {
        return e
    }
    return dao.NewOutboxDaoByDB(tx).Insert(
        "order", "OrderCreated", "order.created", order.ID,
```

逐行讲：为什么 reserve 在 tx 外

- **Redis 不能加入 MySQL 事务**：跨存储两阶段提交业内基本不做，复杂度收益不划算。
- **reserve 失败必须早退**：库存不足 = 业务码 `50001`，不该让 DB 也写一条又回滚。
- **tx 失败要补偿 release**：如果 reserve 成功但 DB 失败，必须把扣下的库存放回 available，否则虚假占用。
- **补偿不是事务的一部分**：即便 `ReleaseReservation` 也失败，DB 已 rollback，库存差额由对账兜底（`GetStockSnapshot` 巡检）。

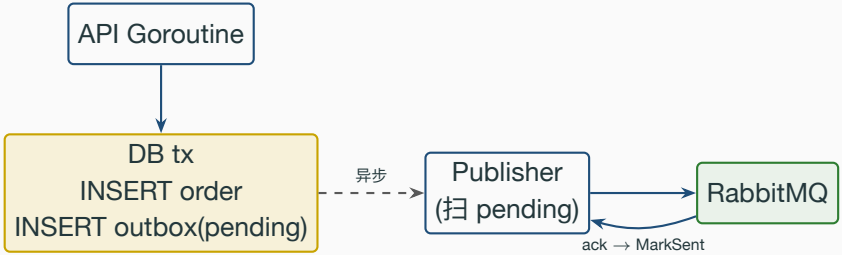
repository/cache/inventory.go:78–125 reserve / commit / release 三脚

逐行讲：为什么 outbox 必须在 tx 内

- 写订单 + 发 MQ 如果**拆成两步**：
  - 写订单成功、发 MQ 前进程挂了 → **下游永远收不到 OrderCreated**。
  - 反过来：先发 MQ 再写 DB，DB 失败 → **幻影订单事件**。
- **outbox 表充当桥梁**：写订单和写 outbox 用同一 `INSERT` 事务，要么都成功要么都没有。
- **异步 publisher** 扫 `status=pending` 的 outbox 行，发 MQ，发完更新成 `sent`。
- 这就是 deck 04 (旧) Outbox 模式在 deck 04 (新) 的落地点。

repository/db/dao/outbox.go:26–41 Insert 同 tx；service/outbox/publisher.go 扫表 publisher

outbox 事件投递时序



业务侧的好处：

API 返回时只承担一次 DB 写代价，MQ 发布失败由 publisher 重试，不阻塞下单链路。

repository/db/dao/outbox.go:44–78 FetchBatch + MarkSent + MarkFailed

同步下单 vs 异步下单

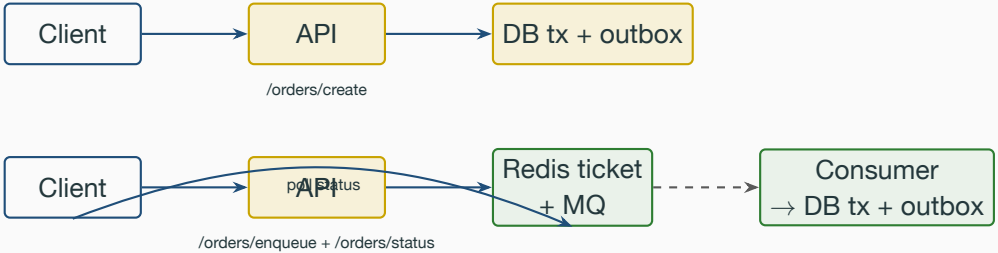
两条下单路并存的业务理由

路径	业务场景	用户体验
`/orders/create`	普通商品 / 流量平稳	一次请求拿到订单号
`/orders/enqueue`	秒杀 / 大促瞬时洪峰	拿 ticket，轮询 status

- 同步：用户等响应的同时 DB 在写；DB 抖动 □ 全部超时。
- 异步：MQ 顶住瞬时压力，consumer 平滑落库；代价是用户多一次轮询。

routes/router.go:92–95 两条路同挂 *Idempotency*

同步 vs 异步：链路对比



service/order_async.go:118–172 enqueue；:175–188 status

OrderEnqueue 核心 12 行

Listing 3: *service/order_async.go:119–150*

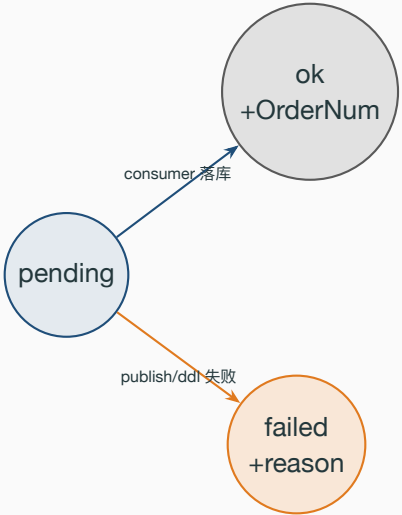
```go
119 func (s *OrderSrv) OrderEnqueue(ctx context.Context,
120     req *types.OrderCreateReq) (resp interface{}, err error) {
121
122     if err = cache.ReserveStock(ctx,
123         req.ProductID, int64(req.Num)); err != nil {
124         return nil, err // 50001 库存不足直接拒
125     }
126
127     ticket := fmt.Sprintf("%d", snowflake.GenSnowflakeID())
128
129     // 把 pending 状态写进 Redis, TTL 1h
130     _ = defaultTicketStore.Put(ctx, ticket,
131         OrderTicketStatus{Status: OrderTicketStatusPending},
132         OrderTicketTTL)
133
134     body, _ := json.Marshal(AsyncOrderTask{Ticket: ticket, ...})
135     return OrderEnqueueResp{Ticket: ticket,
136         Status: OrderTicketStatusPending},
```

逐行讲：ticket 与 TTL 的取舍

- **ticket 用雪花生成**：和订单号同源，全局唯一、客户端不能伪造。
- **TTL = 1h**：覆盖最长支付窗口（15min）+ 客服查单 + 一次客户端崩溃。
- **ticket 不等于订单号**：consumer 落库后回写 `OrderTicketStatus.OrderNum`，客户端读 status 时一并拿到。
- **失败也写 ticket**：投递失败回写 `status=failed, reason=...`，前端不用纠结超时还是真失败。

service/order_async.go:23–52 常量与状态结构

ticket 状态机



- **pending**：API 已收单，consumer 未落库。

用户体验：轮询节奏

- **首查**：API 返回 200 + ticket 后**立刻**打一次 status，多数情况已 ok（端到端 < 100ms）。
- **慢路径**：未 ok 则每 500ms 一次，最多 10 次，覆盖 5s；之后切到 2s/次。
- **超时**：10s 仍 pending □ 提示”系统繁忙，订单稍后可在订单列表查看”，避免一直转圈。
- **不要 long polling**：gomall 单机部署，goroutine hold 住没收益。

service/order_async.go:175–188 OrderStatus；routes/router.go:95 路由

业务码、压测数字、对账

下单链路上的业务码

状态码	含义	客户端动作
50001	库存不足	提示”已售罄”，不进重试
60002	幂等命中	用旧响应，不发新单
70001	限流命中	退避后重试
70002	熔断打开	退避后重试（带原 key）

注意：50001 与 60002 表面都是 200，

但含义**完全相反**：
50001 是”业务上不可能成功”，60002 是”已经成功过了”。
客户端必须按 status 字段分支，不能只看 HTTP code。*pkg/e/code.go:49–58* 业务码定义

压测锚点：50,319 RPS / p95 2.33ms

场景	实测	备注
幂等下单 (`/orders/create`) RPS	**50,319**	50 VU × 15s
幂等下单 p95	2.33 ms	同一 `Idempotency-Key` 重
幂等下单 max	73 ms	
实际写入 DB 的订单数	**1**	75 万次请求

放 75 万次，DB 只多 1 行。
说明：业务上的”重复下单防护”成本接近 0，是因为 60002 直接读 Redis done 态。
order 表的体量 6M 行 / 653 MB 不影响这个延迟——不查表就不付代价。

stressTest/REPORT.md § 各链路压测结果 + § 幂等下单 75 万次重放

对账：reserve / commit / release 不变量

- 库存 Redis 双桶：`available + reserved + sold = 初始水位`。
- DB 订单 + outbox 同事务：两条记录要么都在要么都没有。
- ticket + DB 行：`OrderTicketStatus.Status=ok` $\Longleftrightarrow$ `orders.order_num` 存在。

任何一个不变量被破坏，巡检脚本应当告警：

- Redis 双桶不平 → 库存对账脚本（`GetStockSnapshot`）。
- 订单与 outbox 数量不一致 → 直接 SQL JOIN 抽查。
- ticket=ok 但订单不存在 → consumer 写回路径异常。

repository/cache/inventory.go:128–138 GetStockSnapshot 巡检入口

客服侧：业务码 → 话术

状态码	用户看到	客服话术
50001	”已售罄”	推荐相似商品
60002	自动跳订单详情	”您已下单成功，无需重复”
ticket=failed	红字提示重试	让用户重新提交一次

一句话原则：**业务码不**

暴露技术词，但要给客服**足够区分**的话术。

”幂等命中”和”库存不足”如果都写”系统繁忙”，客服没法定位用户。*deck 10* 治理 *deck* 同款客服话术表，保持业务码一致

回顾与代码路径

把这条链拼回业务侧的一句话

- 购物车 = 凑单/比价/留存的中转，不是技术装饰。
- 地址 = 收货语义的快照，下单瞬间必须冻结。
- 雪花订单号 = 业务（防枚举）+ 性能（顺序写）的合谋。
- OrderCreate 三步：**reserve 在事务外、outbox 在事务内**。
- 同步 / 异步两条路：分别面对” 平稳流量” 和” 瞬时洪峰”。

下单是金融事件——gomall 用”Redis 预扣 + DB 事务 + outbox 桥梁”
把它拆成可证不变量的三步。

面试 Q&A

Q1：为什么 reserve 不放进 DB 事务？

A：跨 Redis + MySQL 的两阶段提交收益不抵复杂度。失败由 ReleaseReservation 补偿，对账兜底。

Q2：outbox 表写满了怎么办？

A：publisher 周期把 sent 行归档；dead 行进单独表人工处理。归档脚本按月按 status 分区清。

Q3：雪花的时钟回拨在 gomall 怎么处理？

A：库内部阻塞等待，单机部署 NTP 用 slew 模式。监控 GenSnowflakeID p99 作为预警。

面试 Q&A（续）

Q4：50001 和 60002 客户端怎么区分对待?

A：50001 终止重试、提示售罄；60002 用旧响应跳订单详情，不重发。

Q5：/orders/enqueue 比 /orders/create 多了什么风险?

A：多一个 ticket 状态需要轮询；consumer 不可用时 ticket 一直 pending 需要 TTL 兜底。

Q6：如果跨店满减要做，购物车要改什么?

A：聚合算价从 user 维度改为 (user, boss_id) 维度，下单也要拆成多店多单。

代码位置一览

加购 / 列表 / 改量 *service/cart.go:29–103*

地址 CRUD *service/address.go:27–127*

- **雪花算法** *pkg/utils/snowflake/snowflake.go:10–25*

同步下单主流程 *service/order.go:40–102*

Redis 预扣库存 *repository/cache/inventory.go:78–95*

- **释放预扣** *repository/cache/inventory.go:113–125*

outbox 同事务写入 *repository/db/dao/outbox.go:26–41*

outbox 扫表发布 *repository/db/dao/outbox.go:44–78*

异步下单 enqueue *service/order_async.go:119–172*

ticket 状态读取 *service/order_async.go:175–188*

- **路由挂载** *routes/router.go:92–95 orders/create + enqueue + status*

配套 deck：04-outbox-saga（Outbox / Saga 机制深挖）、deck 10（流量治理）